

Bài tập về Tái cấu trúc mã nguồn

Bài 1: The smell hunter

Xem xét 4 đoạn code sau:

Đoạn A:

```
public double calculateFee(String t, int h, double r, boolean m) {  
    double f = h * r;  
    if (m) f = f * 0.9;  
    return f;  
}
```

Đoạn B:

```
class UserService {  
    public User findById(int id) { ... }  
    public void sendWelcomeEmail(User user) { ... }  
    public void sendPasswordResetEmail(User user) { ... }  
    public void renderUserProfile(User user) { ... }  
    public String exportUserToCsv(User user) { ... }  
}
```

Đoạn C:

```
public double getArea(String shapeType, double a, double b) {  
    if (shapeType.equals("rectangle")) return a * b;  
    if (shapeType.equals("triangle")) return 0.5 * a * b;  
    if (shapeType.equals("circle")) return 3.14159 * a * a;  
    return -1;  
}
```

Đoạn D:

```
class Report {  
    private String title;  
    private String content;  
    private String authorEmail;
```

```
private String authorName;  
private String authorPhone;  
private String authorAddress;  
}
```

Yêu cầu:

1. Với mỗi đoạn code, xác định loại code smell đang mắc phải, giải thích ngắn gọn tại sao đây là vấn đề và đề xuất kỹ thuật refactor phù hợp.
2. Viết code đã được refactor của mỗi đoạn. Đảm bảo output không thay đổi.

Bài 2: Extract and explain

Phương thức sau tính lương cuối tháng nhưng quá dài và khó đọc:

```
public void printPayroll(String name, double baseSalary,  
                        int workDays, int totalDays,  
                        double taxRate, double bonus) {  
    System.out.println("=== BẢNG LƯƠNG ===");  
    System.out.println("Nhân viên: " + name);  
  
    double actualSalary = baseSalary * workDays / totalDays;  
  
    double insurance = actualSalary * 0.08 + actualSalary * 0.015;  
  
    double taxableIncome = actualSalary - insurance - 11000000;  
    double tax = 0;  
    if (taxableIncome > 0) {  
        if (taxableIncome <= 5000000) tax = taxableIncome * 0.05;  
        else if (taxableIncome <= 10000000) tax = 250000 +  
(taxableIncome - 5000000) * 0.10;  
        else tax = 750000 + (taxableIncome - 10000000) * taxRate;  
    }  
  
    double netSalary = actualSalary - insurance - tax + bonus;  
  
    System.out.println("Lương cơ bản: " + baseSalary);
```

```

        System.out.println("Ngày công: " + workDays + "/" + totalDays);
        System.out.println("Lương thực tế: " + actualSalary);
        System.out.println("Bảo hiểm: " + insurance);
        System.out.println("Thuế TNCN: " + tax);
        System.out.println("Thưởng: " + bonus);
        System.out.println("Thực nhận: " + netSalary);
    }

```

Yêu cầu:

1. Áp dụng **Replace Magic Number with Constant** cho tất cả các hằng số cứng (0.08, 0.015, 11000000, v.v.).
2. Tách các khối tính actualSalary, insurance, tax thành các phương thức riêng với tên rõ nghĩa. Thay actualSalary * 0.08 và actualSalary * 0.015 bằng các biến trung gian có tên tự giải thích.
3. Sau khi refactor, thân hàm **printPayroll()** chỉ được gọi các phương thức con và in kết quả, không chứa logic tính toán trực tiếp.
4. Tạo **main()** sinh dữ liệu mẫu, in output trước và sau refactor để đối chiếu.

Bài 3: Refactor theo từng "small steps"

Cho đoạn code sau:

```

class Vehicle {
    protected String plate;
    protected String brand;
    protected double fuelLevel;    // Chỉ xe chạy xăng mới dùng
    protected int batteryPercent; // Chỉ xe điện mới dùng
}

class MotorBike extends Vehicle {
    public String getInfo() {
        return "Xe máy [" + plate + "] - " + brand;
    }
    public void refuel(double liters) { fuelLevel += liters; }
}

class Car extends Vehicle {

```

```

    public String getInfo() {
        return "Ô tô [" + plate + "] - " + brand;
    }
    public void refuel(double liters) { fuelLevel += liters; }
}

class ElectricCar extends Vehicle {
    public String getInfo() {
        return "Xe điện [" + plate + "] - " + brand;
    }
    public void charge(int percent) { batteryPercent += percent; }
}

```

Yêu cầu:

1. Phân tích vấn đề của fuelLevel và batteryPercent. Đề xuất giải pháp.
2. Phương thức **getInfo()** trong lớp con có vấn đề gì? Đề xuất cách xử lý phù hợp.
3. Thực hiện refactor. Sau đó tạo 3 loại Vehicle rồi gọi genInfo() cho tất cả để kiểm tra output.

Bài 4: Refactor with "small steps"

Đoạn mã sau tính hóa đơn gửi xe, nhưng **receipt()** quá dài, trách nhiệm lẫn lộn, khó mở rộng.

```

import java.util.ArrayList;
import java.util.List;

class Vehicle {
    static final int CAR = 0;
    static final int BIKE = 1;
    static final int TRUCK = 2;

    private final String plate;
    private final int type;

    public Vehicle(String plate, int type) {

```

```

        this.plate = plate;
        this.type = type;
    }
    public String getPlate() { return plate; }
    public int getType() { return type; }
}

```

```

class ParkingTicket {
    private final Vehicle vehicle;
    private final int hours;

    public ParkingTicket(Vehicle vehicle, int hours) {
        this.vehicle = vehicle;
        this.hours = hours;
    }
    public Vehicle getVehicle() { return vehicle; }
    public int getHours() { return hours; }
}

```

```

class ParkingCustomer {
    private final String name;
    private final List<ParkingTicket> tickets = new ArrayList<>();

    public ParkingCustomer(String name) {
        this.name = name;
    }
    public void addTicket(ParkingTicket ticket) {
        tickets.add(ticket);
    }

    public String receipt() {
        double totalFee = 0;
        int bonusPoints = 0;
        String result = "Parking Receipt for " + name + "\n";

        for (ParkingTicket each : tickets) {
            double thisFee = 0;

            // calculate fee per ticket

```

```

switch (each.getVehicle().getType()) {
    case Vehicle.CAR:
        thisFee += 10;
        if (each.getHours() > 2) {
            thisFee += (each.getHours() - 2) * 3;
        }
        break;
    case Vehicle.BIKE:
        thisFee += 5;
        if (each.getHours() > 3) {
            thisFee += (each.getHours() - 3) * 2;
        }
        break;
    case Vehicle.TRUCK:
        thisFee += 15 + each.getHours() * 4;
        break;
}

totalFee += thisFee;

// bonus points
bonusPoints++;
if (each.getVehicle().getType() == Vehicle.TRUCK &&
each.getHours() > 5) {
    bonusPoints++;
}

    result += "\t" + each.getVehicle().getPlate() + "\t" +
thisFee + "\n";
}

    result += "Total fee is " + totalFee + "\n";
    result += "You earned " + bonusPoints + " bonus points";
    return result;
}
}

```

Yêu cầu:

1. Refactor theo **hiều bước nhỏ**, mỗi bước đều giữ nguyên output.

2. Áp dụng ít nhất các kỹ thuật: **Extract Method**, **Move Method**, **Replace Temp with Query**.
3. Kết quả sau refactor:
 - **ParkingCustomer.receipt()** ngắn gọn, chỉ ghép chuỗi.
 - Logic tính phí và điểm thưởng được chuyển sang lớp phù hợp hơn (**ParkingTicket** hoặc **Vehicle**).
4. Tạo **main()** sinh dữ liệu mẫu và in **receipt()** trước/sau refactor để đối chiếu.
5. Tiếp tục áp dụng Replace Conditional with Polymorphism - loại bỏ switch bằng cách tạo các lớp con **Car**, **Bike**, **Truck** kế thừa **Vehicle**, mỗi lớp tự định nghĩa logic tính phí và điểm thưởng.

Bài 5: The delivery calculator

Đoạn code sau tính phí giao hàng dựa trên loại đơn:

```
class Order {
    private String type; // "STANDARD", "EXPRESS", "FRAGILE"
    private double weight;
    private double distance;

    public Order(String type, double weight, double distance) {
        this.type = type; this.weight = weight; this.distance =
distance;
    }

    public double getDeliveryFee() {
        if (type.equals("STANDARD")) {
            return weight * 3000 + distance * 500;
        } else if (type.equals("EXPRESS")) {
            return (weight * 3000 + distance * 500) * 1.5;
        } else if (type.equals("FRAGILE")) {
            return weight * 5000 + distance * 700 + 20000;
        }
        throw new IllegalArgumentException("Loại đơn hàng không hợp
lệ: " + type);
    }
}
```

```

    }

    public String getLabel() {
        if (type.equals("STANDARD")) return "[THƯỜNG]";
        if (type.equals("EXPRESS")) return "[HỎA TỐC]";
        if (type.equals("FRAGILE")) return "[HÀNG DỄ VỠ]";
        return "[KHÔNG XÁC ĐỊNH]";
    }
}

```

Yêu cầu:

1. Phân tích: điều gì xảy ra với đoạn code này nếu cần thêm loại đơn hàng mới?
Cần sửa bao nhiêu chỗ?
2. Áp dụng **Replace Conditional with Polymorphism** để refactor.
3. Sinh dữ liệu chứa ít nhất 4 đơn hàng hỗn hợp, in ra nhãn và phí giao hàng. Đảm bảo output giống hệt trước khi refactor.
4. Thêm loại đơn hàng Bulky (hàng cồng kềnh):
 - phí = weight * 4000 + distance * 600 + 50000
 - nhãn "[HÀNG CỒNG KỀNH]"

So sánh công sức khi thêm loại mới giữa thiết kế cũ và thiết kế mới.

Bài 6: The god class

Lớp **StudentManager** đang đảm nhận quá nhiều trách nhiệm:

```

class StudentManager {
    private String studentId;
    private String name;
    private double gpa;

    private String courseId;
    private String courseName;
    private int credits;

    private double midtermScore;
}

```



```

private double finalScore;
private double assignmentScore;

public double calculateFinalGrade() {
    return assignmentScore * 0.2 + midtermScore * 0.3 +
finalScore * 0.5;
}

public String getAcademicStatus() {
    double grade = calculateFinalGrade();
    if (grade >= 8.5) return "Giỏi";
    if (grade >= 7.0) return "Khá";
    if (grade >= 5.5) return "Trung bình";
    return "Yếu";
}

public void printTranscript() {
    System.out.println("Sinh viên: " + name + " (" + studentId +
    ")");
    System.out.println("Môn học: " + courseName + " (" + courseId
    + ") - " + credits + " tín chỉ");
    System.out.println("Điểm GK: " + midtermScore + " | Điểm CK:
    " + finalScore
        + " | Điểm BT: " + assignmentScore);
    System.out.printf("Điểm tổng kết: %.1f - Học lực: %s\n",
        calculateFinalGrade(),
        getAcademicStatus());
}
}

```

Yêu cầu:

1. Liệt kê rõ từng nhiệm vụ của mà lớp **StudentManager** đang đảm nhận.
2. Áp dụng **Extract Class** để refactor lớp **StudentManager**. Viết lại **printTranscript()** phù hợp với thiết kế mới và đảm bảo output giống hệt code gốc.
3. Giả sử hệ thống cần thêm lớp **TeachingAssitant** (trợ giảng) - cũng có id và name nhưng không có gpa. Đề xuất giải pháp phù hợp.

Bài tập về Kiểm thử

Bài 1: The discount inspector

Cho hàm `calculateDiscount(double price, String memberType)` với đặc tả sau:

- `price < 0`: ném `IllegalArgumentException`
- `memberType` là `GUEST`: không có chiết khấu (0%)
- `memberType` là `MEMBER`: chiết khấu 5% nếu `price < 100`, chiết khấu 10% nếu `price >= 100`
- `memberType` là `VIP`: chiết khấu 15% nếu `price < 100`, chiết khấu 20% nếu `price >= 100`
- `memberType` khác: ném `IllegalArgumentException`

Yêu cầu:

1. Xác định các lớp tương đương (equivalence class) cho tham số `price`.
2. Lập bảng test case theo EP: mỗi lớp tương đương có ít nhất một test case đại diện. Bảng gồm các cột: **Mã TC** | **Mô tả** | **price** | **memberType** | **Kết quả mong đợi**.
3. Áp dụng BVA cho tham số `price` tại ranh giới `price = 0` và `price = 100`. Liệt kê đầy đủ các giá trị biên cần test (`min-`, `min`, `min+`, `max-`, `max`, `max+`).
4. Áp dụng **2-way combinatorial** testing cho cặp (`price`, `memberType`). Thiết kế bộ test tối thiểu đảm bảo mọi cặp giá trị đều xuất hiện trong ít nhất một test case.

Bài 2: The first JUnit

Cho đoạn code sau:

```
public class MathUtils {
    public static int max(int a, int b) {
        if (a >= b) return a;
        return b;
    }

    public static int divide(int a, int b) {
        if (b == 0) {
```

```

        throw new IllegalArgumentException("Divider must not be
zero");
    }
    return a / b;
}
}

```

Yêu cầu:

- Thiết kế test case cho `max(int a, int b)`:
 - Áp dụng EP: xác định các lớp tương đương ($a > b$, $a = b$, $a < b$).
 - Áp dụng BVA: bổ sung các giá trị biên tại `Integer.MIN_VALUE` và `Integer.MAX_VALUE`.
- Thiết kế test case cho `divide(int a, int b)`:
 - Áp dụng EP: xác định các lớp tương đương ($b > 0$, $b < 0$, $b = 0$).
 - Đảm bảo có test case kiểm tra ngoại lệ khi $b = 0$.
- Cài đặt tất cả test case vào class `MathUtilsTest` sử dụng JUnit.
- Thêm `@BeforeAll` để in "=== Bắt đầu chạy MathUtilsTest ===" và `@AfterAll` để in "=== Kết thúc ===". Giải thích tại sao phương thức được đánh dấu `@BeforeAll` bắt buộc phải là static.

Bài 3: The buggy trap

Cho đoạn code sau

```

public class GradeClassifier {

    /**
     * Phân loại học lực dựa trên điểm GPA (thang 10).
     * [0.0, 5.0) → "Yếu"
     * [5.0, 6.5) → "Trung bình"
     * [6.5, 8.0) → "Khá"
     * [8.0, 10.0] → "Giỏi"
     * Ngoài [0.0, 10.0]: ném IllegalArgumentException
     */
    public static String classifyGrade(double gpa) {
        if (gpa < 0.0 || gpa > 10.0) {
            throw new IllegalArgumentException("GPA không hợp lệ: " +
gpa);

```

```

    }
    if (gpa <= 5.0) return "Yếu";
    if (gpa <= 6.5) return "Trung bình";
    if (gpa < 8.0) return "Khá";
    return "Giỏi";
}
}

```

Yêu cầu:

1. Chỉ dựa vào Javadoc, thiết kế bộ test case bằng EP và BVA.
2. Viết class **GradeClassifierTest** với JUnit, cài đặt tất cả test case vừa thiết kế.
3. Chạy và quan sát: test case nào bị FAIL? Ghi lại giá trị đầu vào, kết quả thực tế, kết quả mong đợi. Từ các test FAIL, suy luận: lỗi nằm ở đâu? Mô tả lỗi bằng lời mà không cần đọc lại code.
4. Sửa hàm **classifyGrade** để tất cả test đều PASS.
5. Sau khi sửa xong, thêm test kiểm tra ngoại lệ cho `gpa = -0.1` và `gpa = 10.1` bằng `assertThrows`. Đảm bảo kiểm tra cả nội dung thông báo lỗi.

Bài 4: The bank account tester

Cho đoạn code sau:

```

public class BankAccount {
    private final String accountNumber;
    private String ownerName;
    private double balance;

    public BankAccount(String accountNumber, String ownerName) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = 0.0;
    }

    public BankAccount(String accountNumber, String ownerName, double
initialBalance) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        if (initialBalance < 0) {

```

```

        System.err.println("Số dư ban đầu không hợp lệ. Gán mặc
định là 0.");
        this.balance = 0.0;
    } else {
        this.balance = initialBalance;
    }
}

public void deposit(double amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("Số tiền nạp phải lớn
hơn 0.");
    }
    this.balance += amount;
}

public boolean withdraw(double amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("Số tiền rút phải lớn
hơn 0.");
    }
    if (amount > this.balance) {
        return false;
    }
    this.balance -= amount;
    return true;
}

public double getBalance() {
    return balance;
}

public String getAccountNumber() {
    return accountNumber;
}
}

```

Yêu cầu:

1. Thiết kế test case bằng EP và BVA cho **deposit(double amount)** và

`withdraw(double amount).`

2. Cài đặt toàn bộ test case bằng JUnit với số dư ban đầu là 500 trước mỗi test.
3. Viết một test method kiểm tra tính nhất quán theo trình tự: số dư ban đầu là 0 → nạp 500 → rút 200 (thành công) → rút 400 (thất bại) → kiểm tra số dư cuối phải đúng bằng 300.